Tecnológico Nacional De México

Campus Delicias

INGENIERÍA EN SISTEMAS COMPUTACIONALES

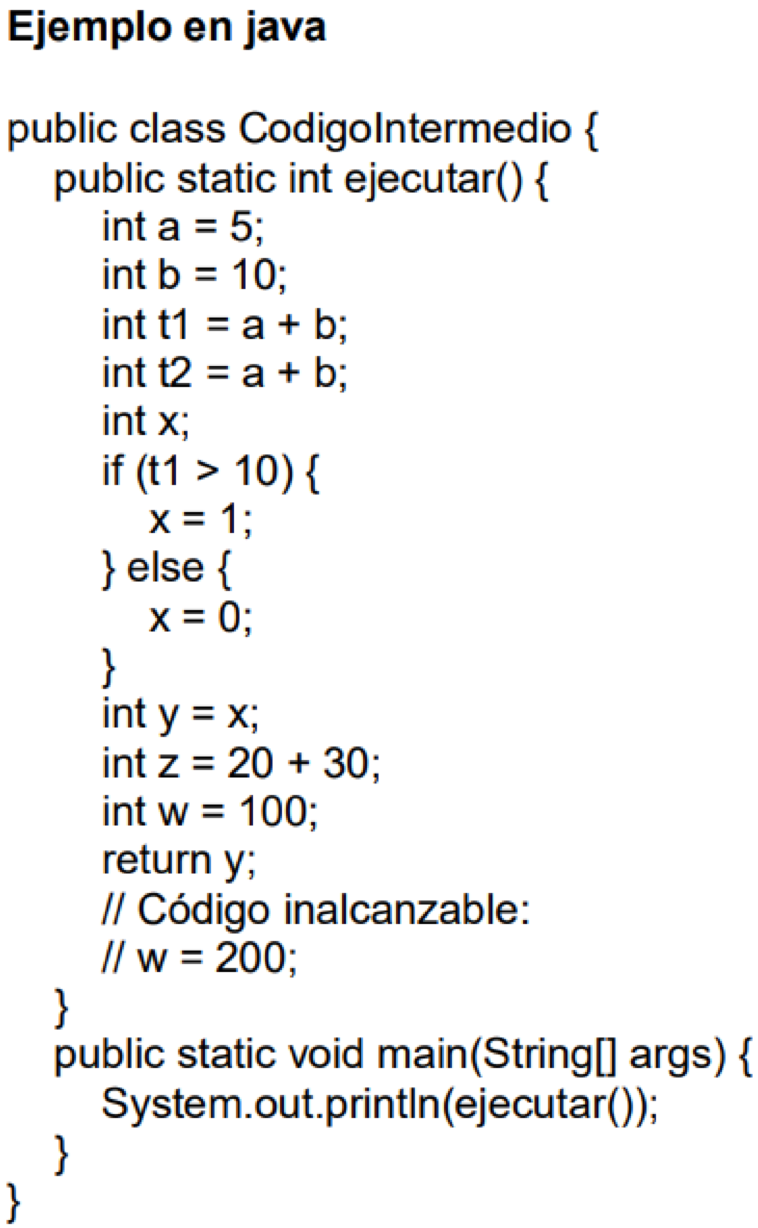
Lenguajes y automatas II

Entregable unidad 3

Alumno: César Moriel Esparza

Docente: Alicia Robles Ruiz

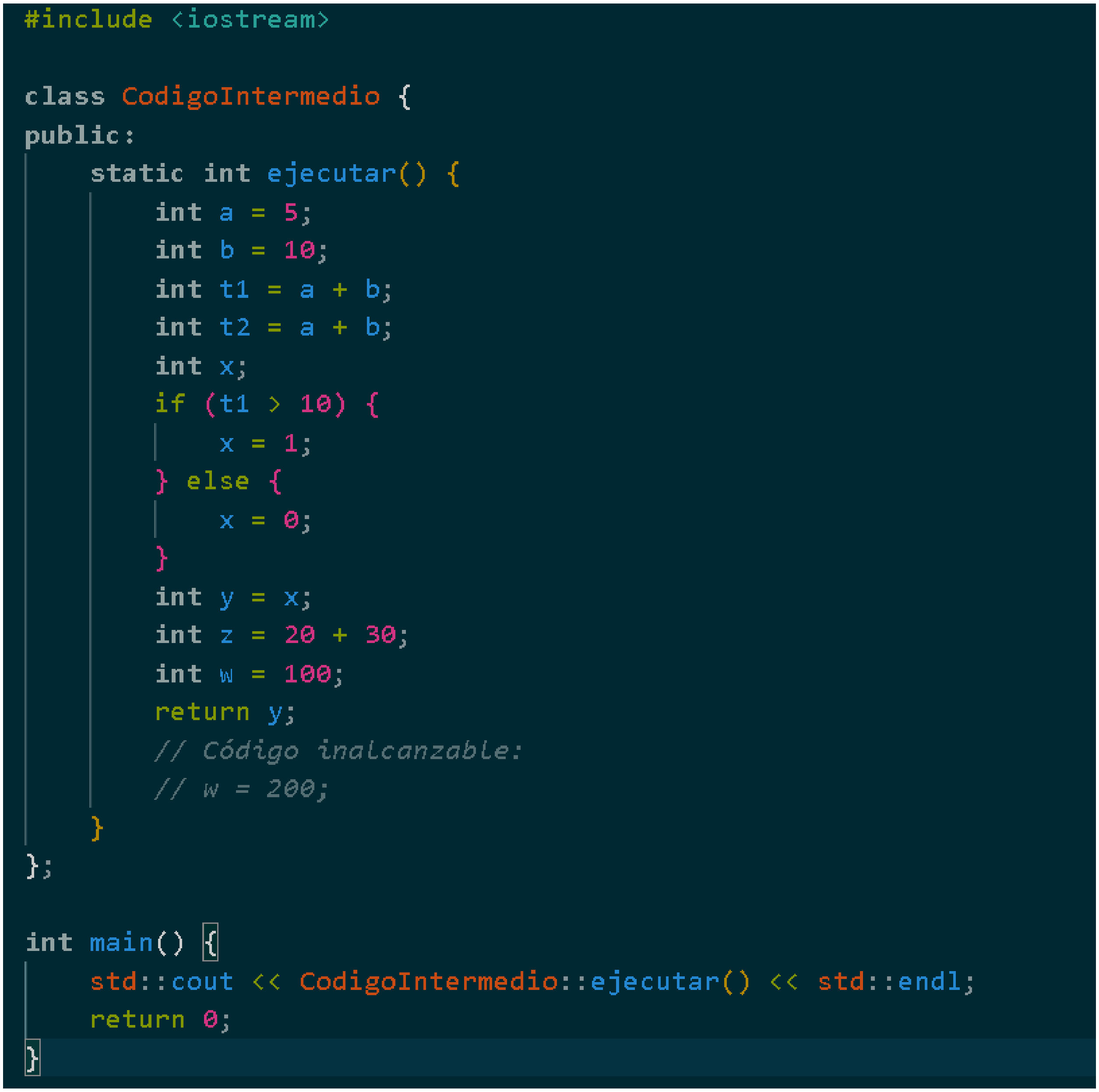
A continuación en el presente documento se presentarán las evidencias del compilador previamente revisado en clase con el código de ejemplo que se proporcionó, el cual es el siguiente:



Ejemplo en java

```
public class CodigoIntermedio {
    public static int ejecutar() {
        int a = 5;
        int b = 10;
        int t1 = a + b;
        int t2 = a + b;
        int x;
        if (t1 > 10) {
            x = 1;
        } else {
            x = 0;
        }
        int y = x;
        int z = 20 + 30;
        int w = 100;
        return y;
        // Código inalcanzable:
        // w = 200;
    }
    public static void main(String[] args) {
        System.out.println(ejecutar());
    }
}
```

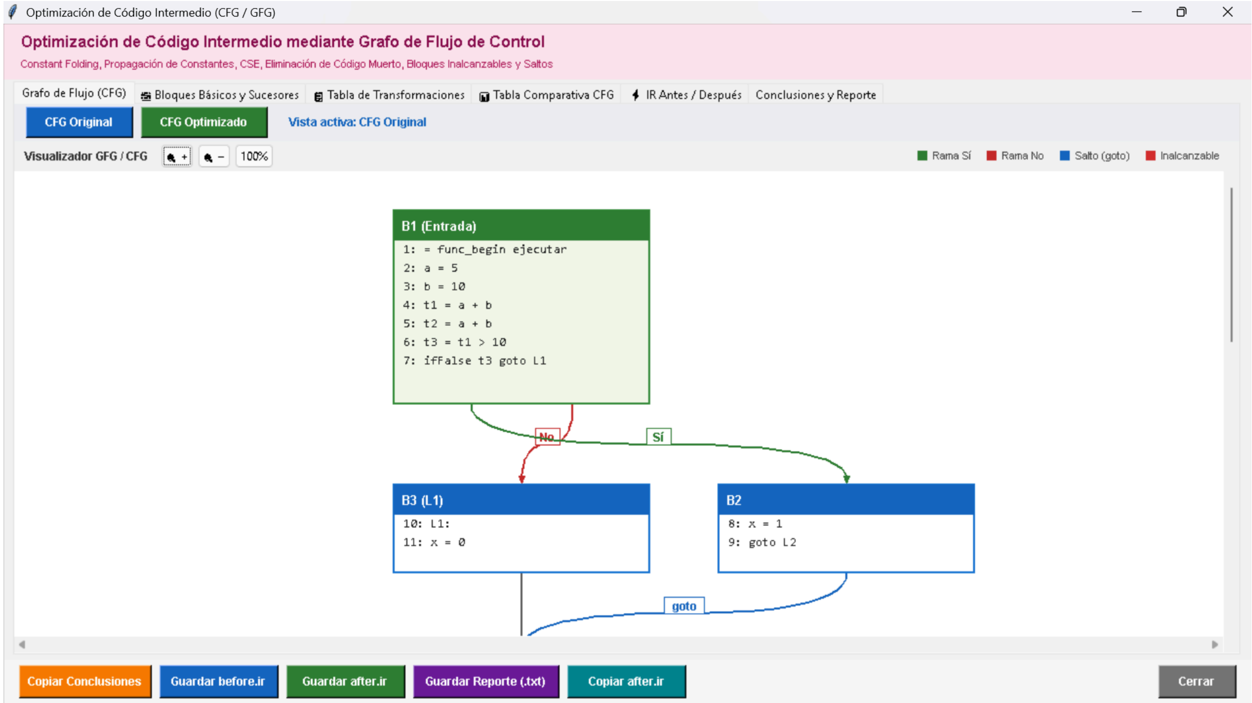
Como el presente compilador está construido para analizar y en este caso optimizar el código en c++ el código traducido a dicho lenguaje es este:



```cpp
#include <iostream>

class CodigoIntermedio {
public:
    static int ejecutar() {
        int a = 5;
        int b = 10;
        int t1 = a + b;
        int t2 = a + b;
        int x;
        if (t1 > 10) {
            x = 1;
        } else {
            x = 0;
        }
        int y = x;
        int z = 20 + 30;
        int w = 100;
        return y;
        // Código inalcanzable:
        // w = 200;
    }
};

int main() {
    std::cout << CodigoIntermedio::ejecutar() << std::endl;
    return 0;
}
```

Por consiguiente el resultado de optimizar este código es el siguiente:

Grafo de flujo original:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
CFG Original
CFG Optimizado
Vista activa: CFG Original
Visualizador GFG / CFG
+
−
100%
🟩 Rama Sí
🟥 Rama No
🟦 Salto (goto)
🟥 Inalcanzable
B1 (Entrada)
No
Sí
B3 (L1)
B2
goto
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

B1 (Entrada)

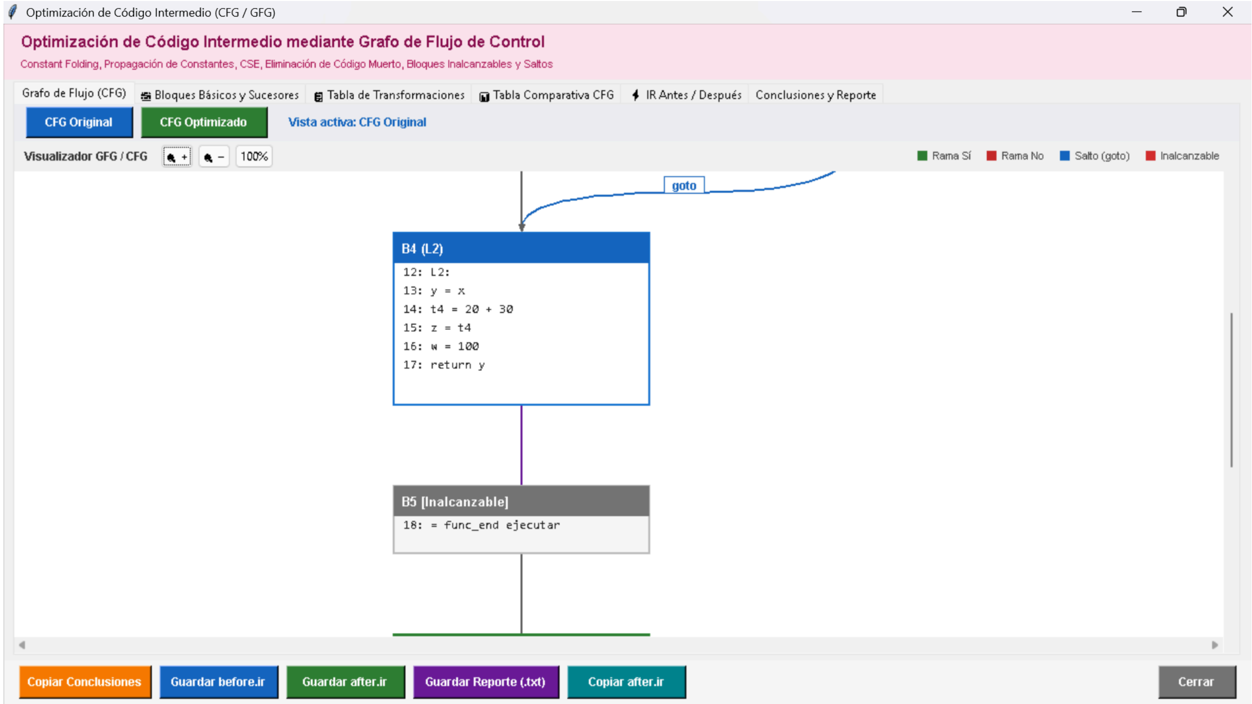
```
1: = func_begin ejecutar
2: a = 5
3: b = 10
4: t1 = a + b
5: t2 = a + b
6: t3 = t1 > 10
7: ifFalse t3 goto L1
```

B3 (L1)

```
10: L1:
11: x = 0
```

B2

```
8: x = 1
9: goto L2
```

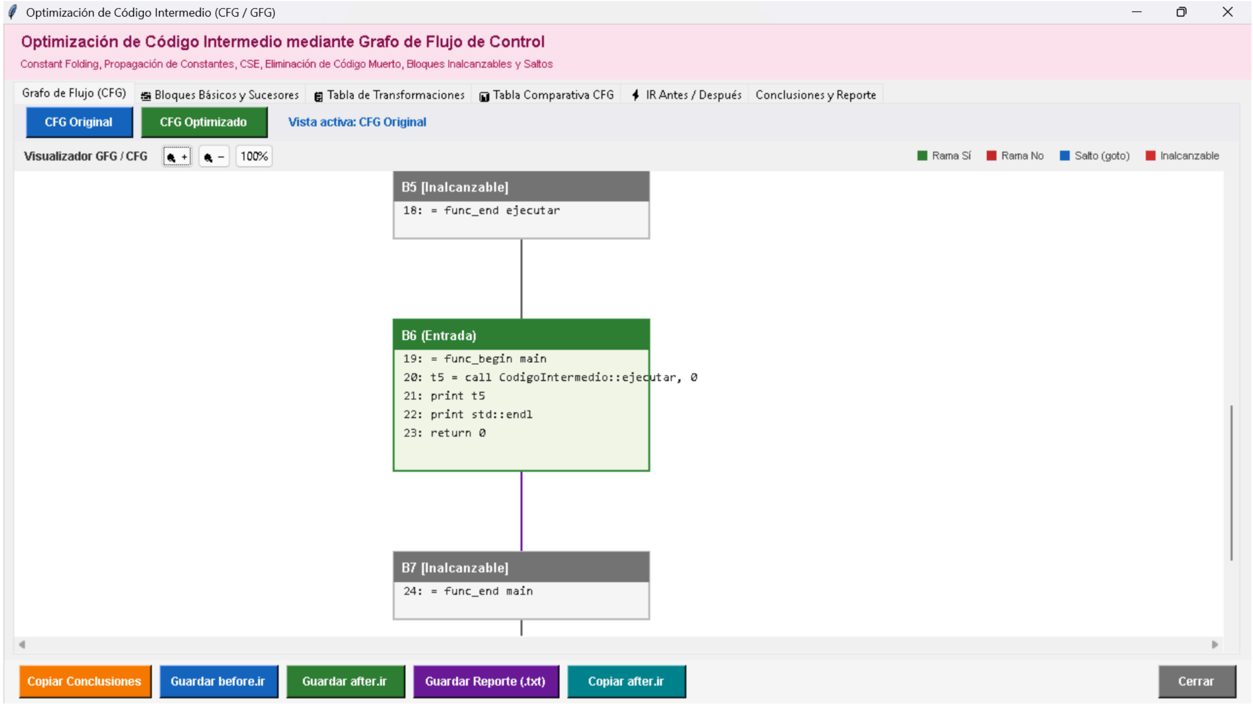

Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
CFG Original
CFG Optimizado
Vista activa: CFG Original
Visualizador GFG / CFG
+
−
100%
🟩 Rama Sí
🟥 Rama No
🟦 Salto (goto)
🟥 Inalcanzable
goto
B4 (L2)
B5 [Inalcanzable]
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

B4 (L2)

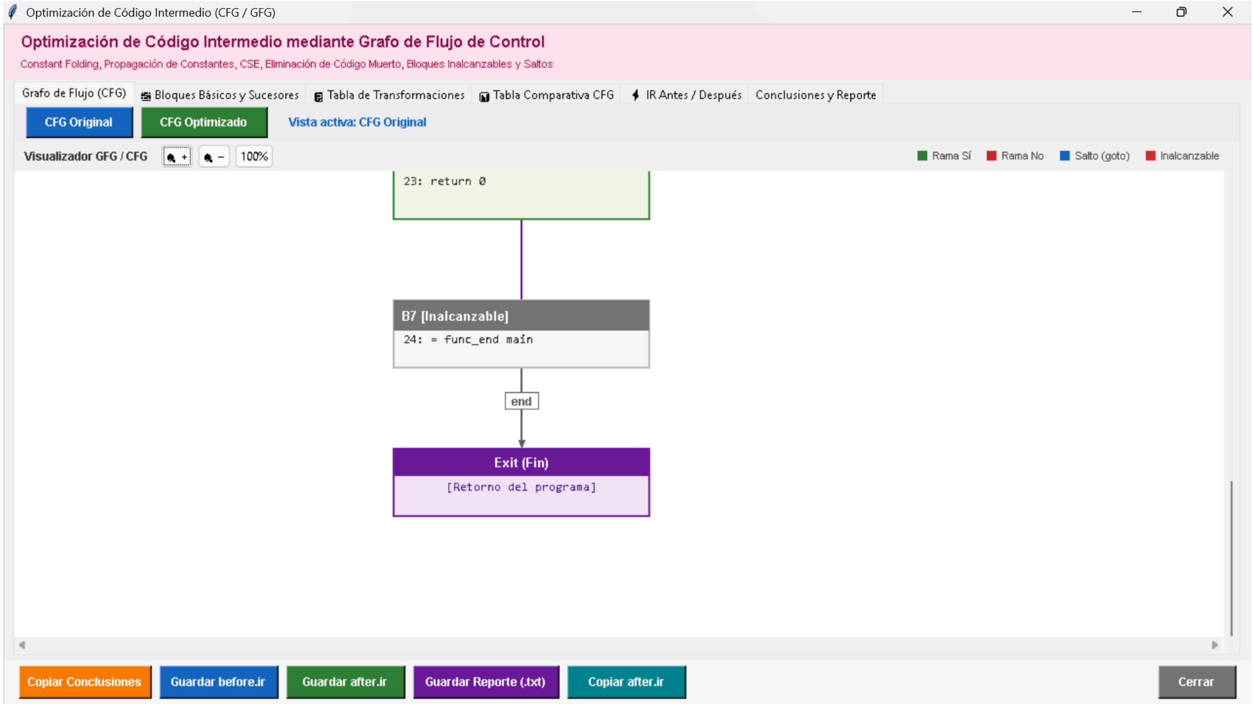
```
12: L2:
13: y = x
14: t4 = 20 + 30
15: z = t4
16: w = 100
17: return y
```

B5 [Inalcanzable]

```
18: = func_end ejecutar
```

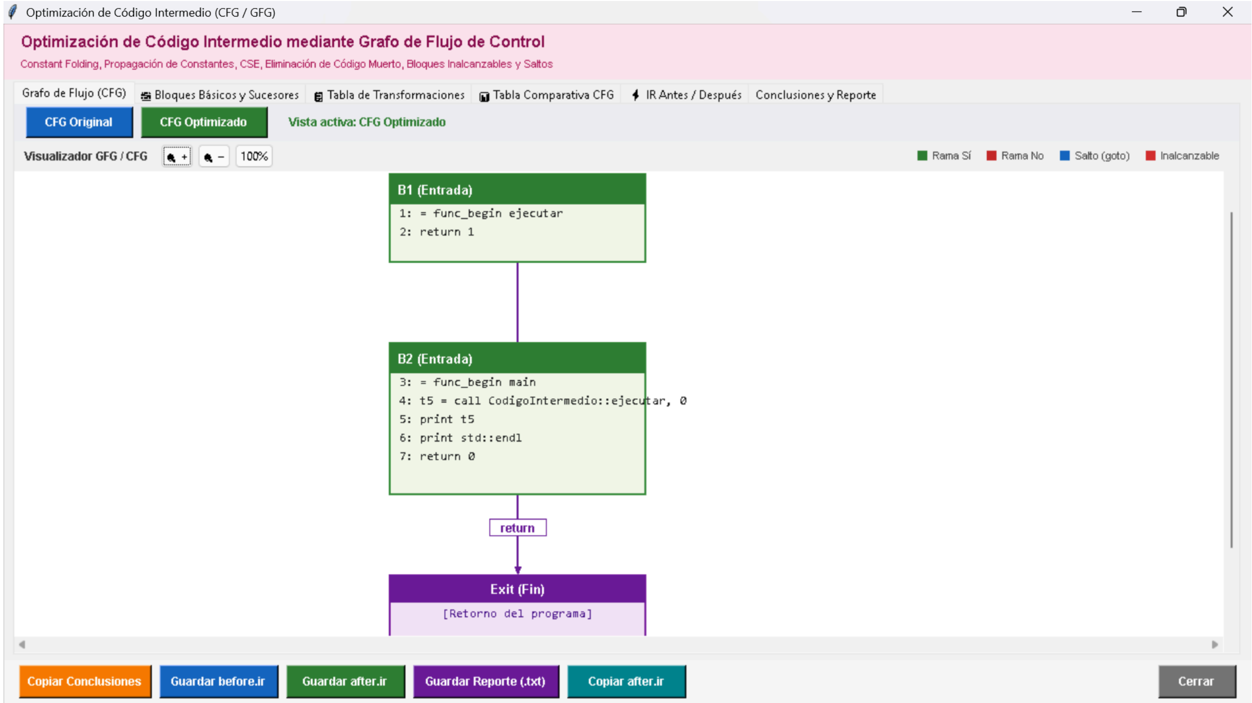

Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
CFG Original
CFG Optimizado
Vista activa: CFG Original
Visualizador GFG / CFG
100%
Rama Sí
Rama No
Salto (goto)
Inalcanzable
B5 [Inalcanzable]
B6 (Entrada)
B7 [Inalcanzable]
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

```
B5 [Inalcanzable]
18: = func_end ejecutar

B6 (Entrada)
19: = func_begin main
20: t5 = call CodigoIntermedio::ejecutar, 0
21: print t5
22: print std::endl
23: return 0

B7 [Inalcanzable]
24: = func_end main
```


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
CFG Original
CFG Optimizado
Vista activa: CFG Original
Visualizador GFG / CFG
100%
Rama Sí
Rama No
Salto (goto)
Inalcanzable
B7 [Inalcanzable]
end
Exit (Fin)
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

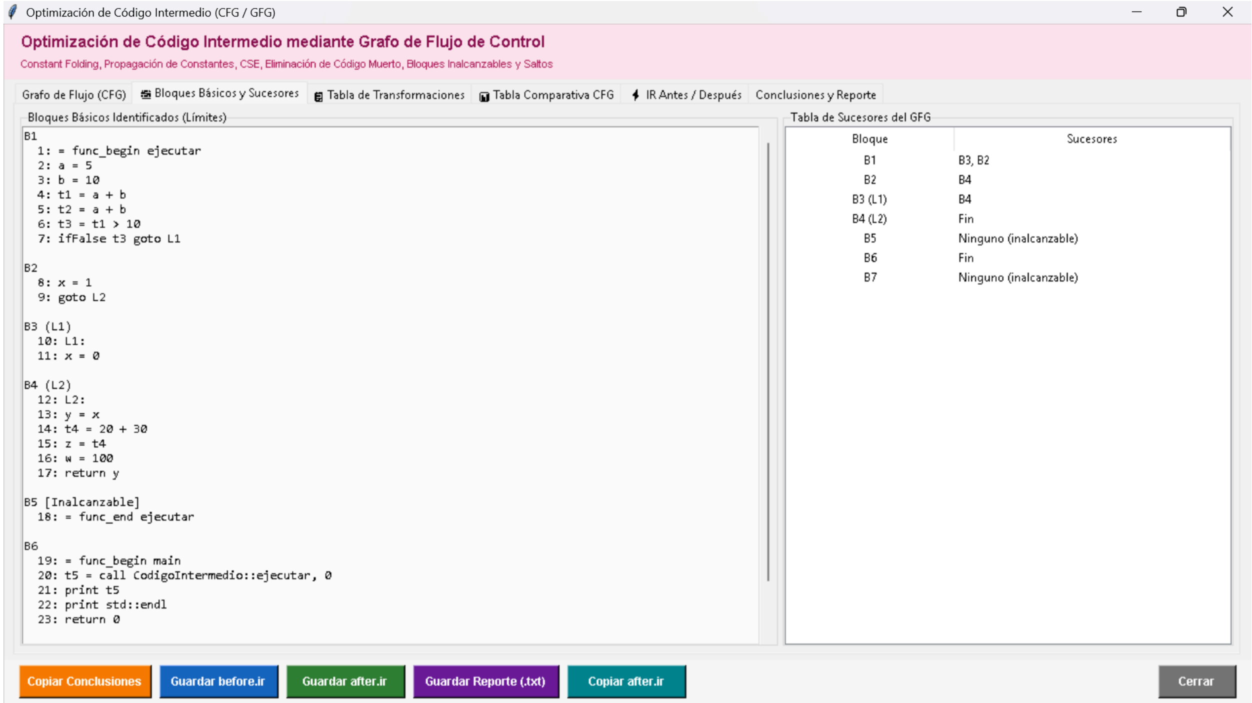
```
23: return 0

B7 [Inalcanzable]
24: = func_end main

Exit (Fin)
[Retorno del programa]
```

Grafo de flujo optimizado:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
CFG Original
CFG Optimizado
Vista activa: CFG Optimizado
Visualizador GFG / CFG
100%
Rama Sí
Rama No
Salto (goto)
Inalcanzable
B1 (Entrada)
B2 (Entrada)
return
Exit (Fin)
[Retorno del programa]
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

```
B1 (Entrada)
1: = func_begin ejecutar
2: return 1

B2 (Entrada)
3: = func_begin main
4: t5 = call CodigoIntermedio::ejecutar, 0
5: print t5
6: print std::endl
7: return 0
```

Bloques básicos y sucesores:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

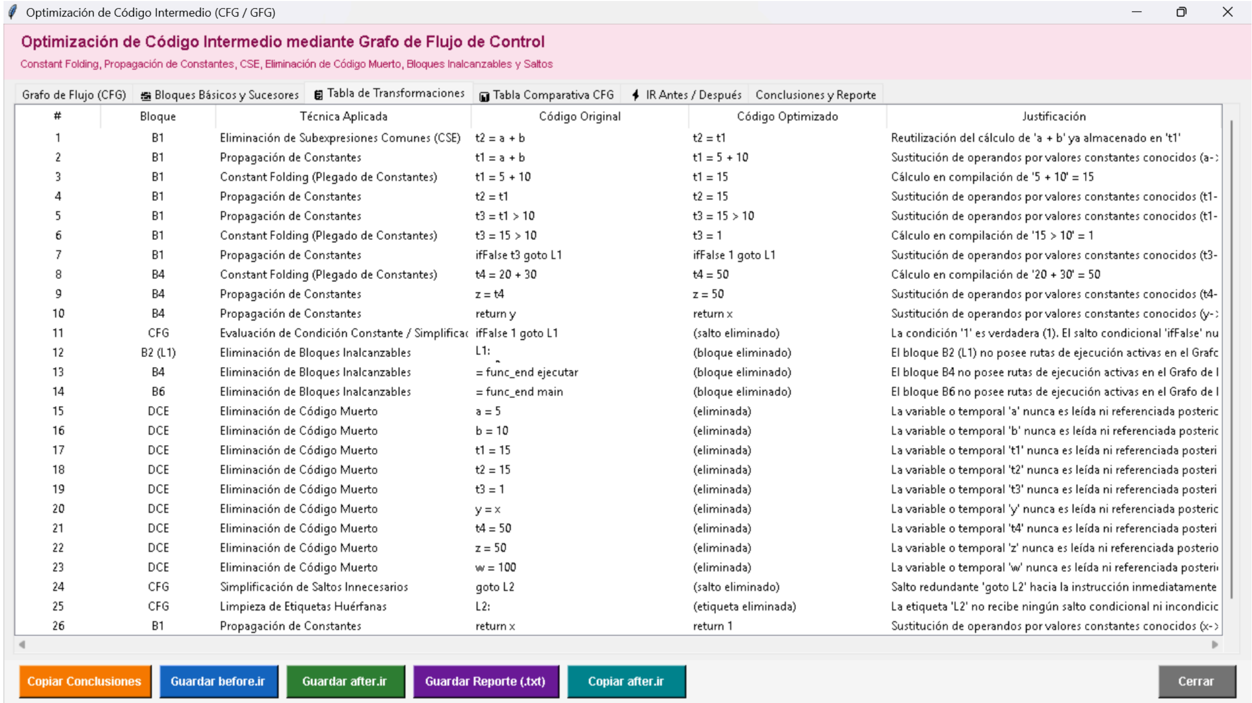
Bloques Básicos Identificados (Límites)

```
B1
  1: = func_begin ejecutar
  2: a = 5
  3: b = 10
  4: t1 = a + b
  5: t2 = a + b
  6: t3 = t1 > 10
  7: ifFalse t3 goto L1

B2
  8: x = 1
  9: goto L2

B3 (L1)
  10: L1:
  11: x = 0

B4 (L2)
  12: L2:
  13: y = x
  14: t4 = 20 + 30
  15: z = t4
  16: w = 100
  17: return y

B5 [Inalcanzable]
  18: = func_end ejecutar

B6
  19: = func_begin main
  20: t5 = call CodigoIntermedio::ejecutar, 0
  21: print t5
  22: print std::endl
  23: return 0
```

Tabla de Sucesores del GFG

Bloque	Sucesores
B1	B3, B2
B2	B4
B3 (L1)	B4
B4 (L2)	Fin
B5	Ninguno (inalcanzable)
B6	Fin
B7	Ninguno (inalcanzable)

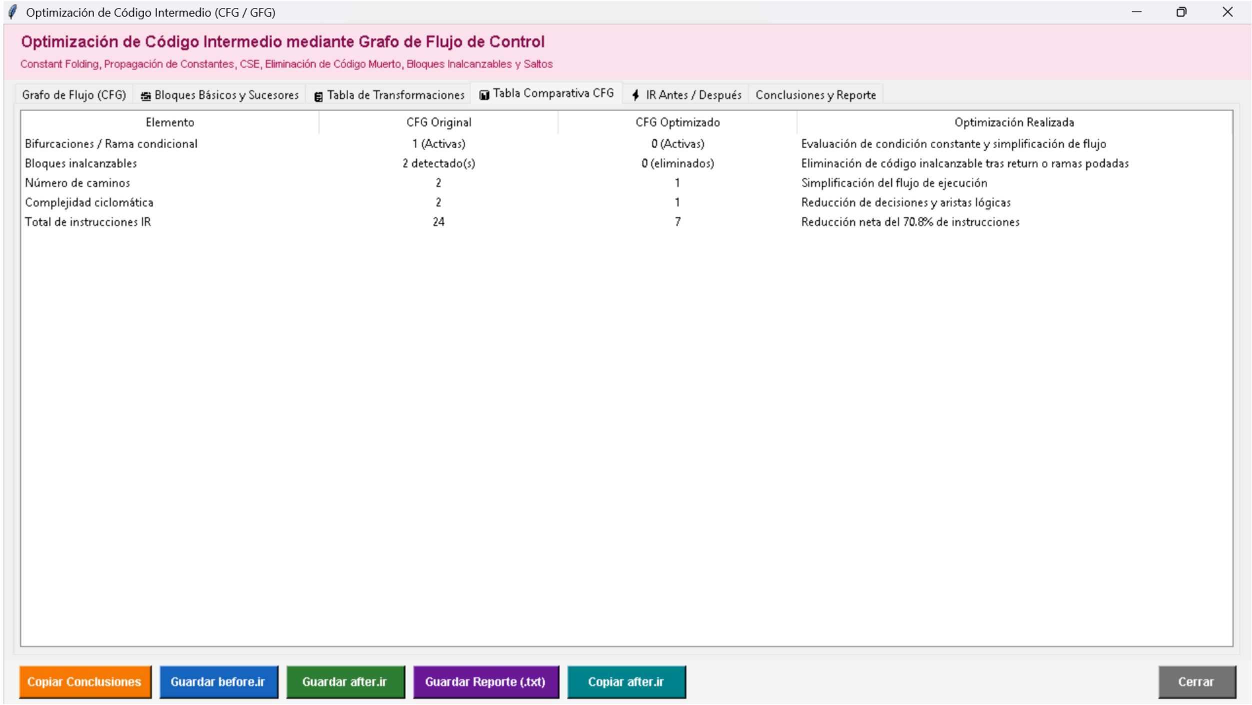
Tabla de transformaciones


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

#	Bloque	Técnica Aplicada	Código Original	Código Optimizado	Justificación
1	B1	Eliminación de Subexpresiones Comunes (CSE)	t2 = a + b	t2 = t1	Reutilización del cálculo de 'a + b' ya almacenado en 't1'
2	B1	Propagación de Constantes	t1 = a + b	t1 = 5 + 10	Sustitución de operandos por valores constantes conocidos (a-:
3	B1	Constant Folding (Plegado de Constantes)	t1 = 5 + 10	t1 = 15	Cálculo en compilación de '5 + 10' = 15
4	B1	Propagación de Constantes	t2 = t1	t2 = 15	Sustitución de operandos por valores constantes conocidos (t1-
5	B1	Propagación de Constantes	t3 = t1 > 10	t3 = 15 > 10	Sustitución de operandos por valores constantes conocidos (t1-
6	B1	Constant Folding (Plegado de Constantes)	t3 = 15 > 10	t3 = 1	Cálculo en compilación de '15 > 10' = 1
7	B1	Propagación de Constantes	ifFalse t3 goto L1	ifFalse 1 goto L1	Sustitución de operandos por valores constantes conocidos (t3-
8	B4	Constant Folding (Plegado de Constantes)	t4 = 20 + 30	t4 = 50	Cálculo en compilación de '20 + 30' = 50
9	B4	Propagación de Constantes	z = t4	z = 50	Sustitución de operandos por valores constantes conocidos (t4-
10	B4	Propagación de Constantes	return y	return x	Sustitución de operandos por valores constantes conocidos (y-:
11	CFG	Evaluación de Condición Constante / Simplificac	ifFalse 1 goto L1	(salto eliminado)	La condición '1' es verdadera (1). El salto condicional 'ifFalse' nu
12	B2 (L1)	Eliminación de Bloques Inalcanzables	L1:	(bloque eliminado)	El bloque B2 (L1) no posee rutas de ejecución activas en el Grafc
13	B4	Eliminación de Bloques Inalcanzables	= func_end ejecutar	(bloque eliminado)	El bloque B4 no posee rutas de ejecución activas en el Grafo de l
14	B6	Eliminación de Bloques Inalcanzables	= func_end main	(bloque eliminado)	El bloque B6 no posee rutas de ejecución activas en el Grafo de l
15	DCE	Eliminación de Código Muerto	a = 5	(eliminada)	La variable o temporal 'a' nunca es leída ni referenciada posteric
16	DCE	Eliminación de Código Muerto	b = 10	(eliminada)	La variable o temporal 'b' nunca es leída ni referenciada posteric
17	DCE	Eliminación de Código Muerto	t1 = 15	(eliminada)	La variable o temporal 't1' nunca es leída ni referenciada posteri
18	DCE	Eliminación de Código Muerto	t2 = 15	(eliminada)	La variable o temporal 't2' nunca es leída ni referenciada posteri
19	DCE	Eliminación de Código Muerto	t3 = 1	(eliminada)	La variable o temporal 't3' nunca es leída ni referenciada posteri
20	DCE	Eliminación de Código Muerto	y = x	(eliminada)	La variable o temporal 'y' nunca es leída ni referenciada posteric
21	DCE	Eliminación de Código Muerto	t4 = 50	(eliminada)	La variable o temporal 't4' nunca es leída ni referenciada posteri
22	DCE	Eliminación de Código Muerto	z = 50	(eliminada)	La variable o temporal 'z' nunca es leída ni referenciada posterio
23	DCE	Eliminación de Código Muerto	w = 100	(eliminada)	La variable o temporal 'w' nunca es leída ni referenciada posteri
24	CFG	Simplificación de Saltos Innecesarios	goto L2	(salto eliminado)	Salto redundante 'goto L2' hacia la instrucción inmediatamente
25	CFG	Limpieza de Etiquetas Huérfanas	L2:	(etiqueta eliminada)	La etiqueta 'L2' no recibe ningún salto condicional ni incondicic
26	B1	Propagación de Constantes	return x	return 1	Sustitución de operandos por valores constantes conocidos (x->

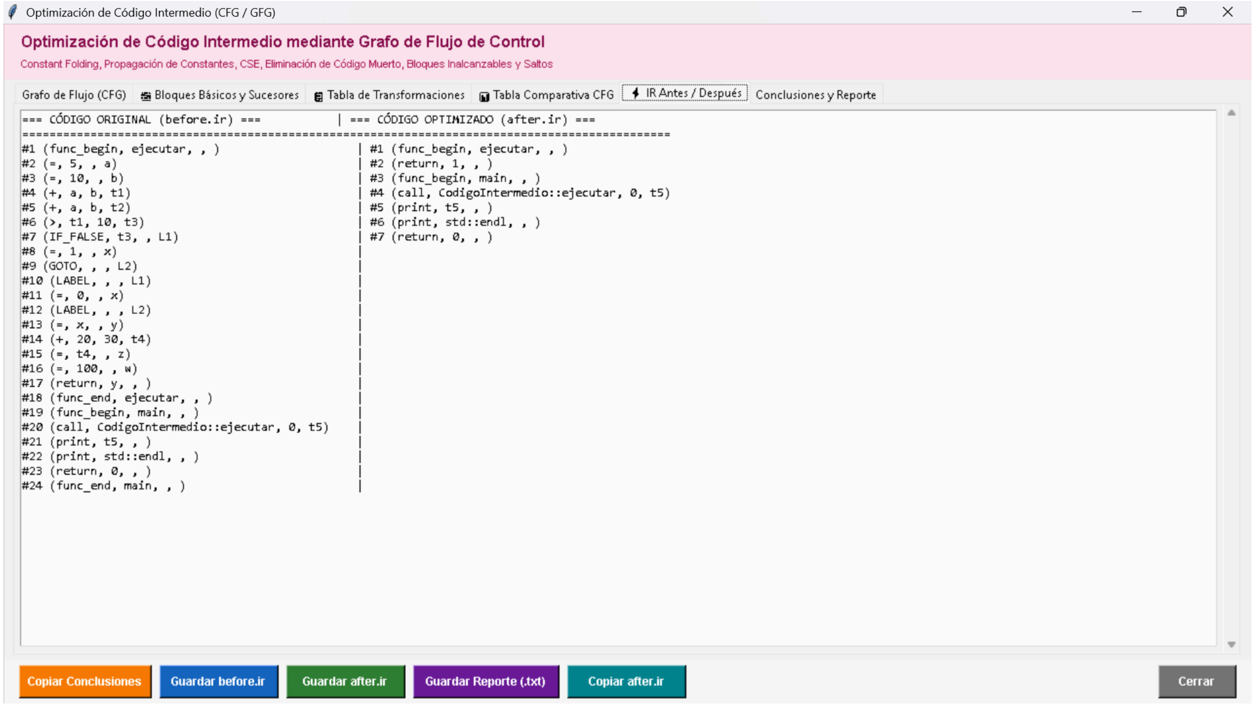
27	DCE	Eliminación de Código Muerto	x = 1	(eliminada)	La variable o temporal 'x' nunca es leída ni referenciada posterio

Tabla de comparación entre grafos de flujo:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

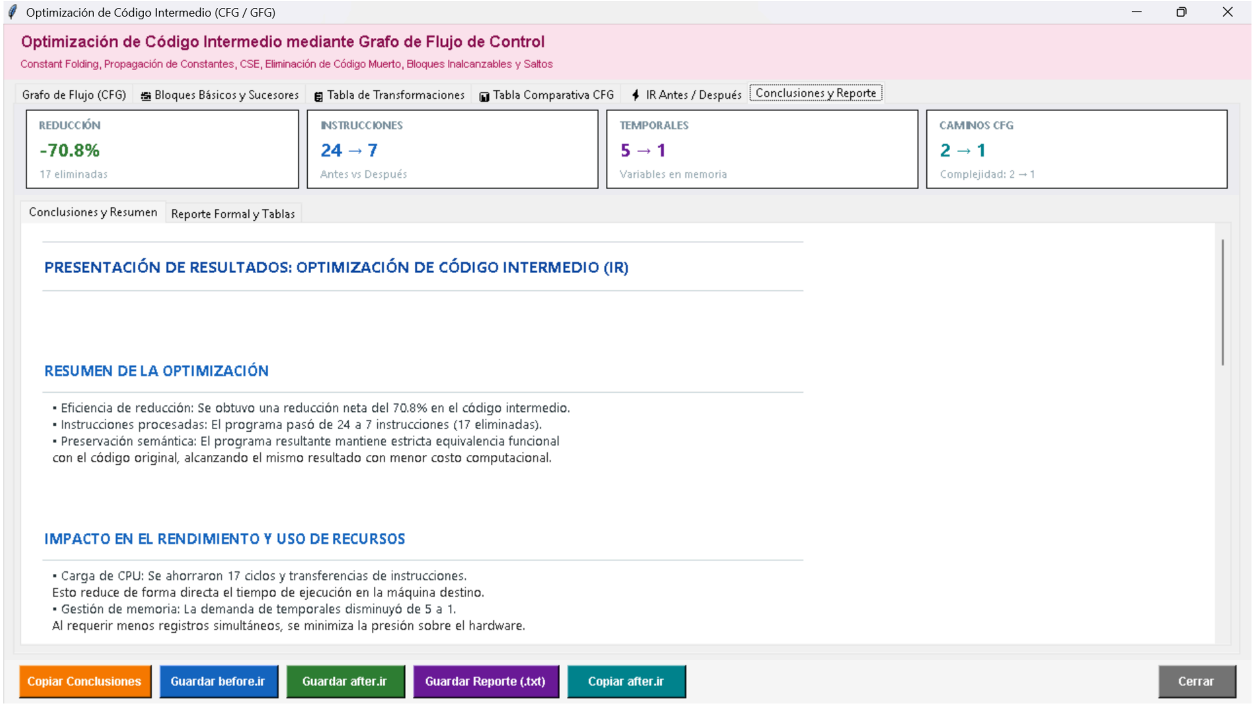
Elemento	CFG Original	CFG Optimizado	Optimización Realizada
Bifurcaciones / Rama condicional	1 (Activas)	0 (Activas)	Evaluación de condición constante y simplificación de flujo
Bloques inalcanzables	2 detectado(s)	0 (eliminados)	Eliminación de código inalcanzable tras return o ramas podadas
Número de caminos	2	1	Simplificación del flujo de ejecución
Complejidad ciclomática	2	1	Reducción de decisiones y aristas lógicas
Total de instrucciones IR	24	7	Reducción neta del 70.8% de instrucciones

Comparación de los códigos ir:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

```
=== CÓDIGO ORIGINAL (before.ir) ===           | === CÓDIGO OPTIMIZADO (after.ir) ===
===============================================================================================
#1 (func_begin, ejecutar, , )                    | #1 (func_begin, ejecutar, , )
#2 (=, 5, , a)                                   | #2 (return, 1, , )
#3 (=, 10, , b)                                  | #3 (func_begin, main, , )
#4 (+, a, b, t1)                                 | #4 (call, CodigoIntermedio::ejecutar, 0, t5)
#5 (+, a, b, t2)                                 | #5 (print, t5, , )
#6 (>, t1, 10, t3)                               | #6 (print, std::endl, , )
#7 (IF_FALSE, t3, , L1)                          | #7 (return, 0, , )
#8 (=, 1, , x)                                   |
#9 (GOTO, , , L2)                                |
#10 (LABEL, , , L1)                              |
#11 (=, 0, , x)                                  |
#12 (LABEL, , , L2)                              |
#13 (=, x, , y)                                  |
#14 (+, 20, 30, t4)                              |
#15 (=, t4, , z)                                 |
#16 (=, 100, , w)                                |
#17 (return, y, , )                              |
#18 (func_end, ejecutar, , )                     |
#19 (func_begin, main, , )                       |
#20 (call, CodigoIntermedio::ejecutar, 0, t5)    |
#21 (print, t5, , )                              |
#22 (print, std::endl, , )                       |
#23 (return, 0, , )                              |
#24 (func_end, main, , )                         |
```

Conclusiones y reporte:


Optimización de Código Intermedio (CFG / GFG)
Optimización de Código Intermedio mediante Grafo de Flujo de Control
Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Conclusiones y Resumen
Reporte Formal y Tablas
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

REDUCCIÓN	INSTRUCCIONES	TEMPORALES	CAMINOS CFG
-70.8%	**24 → 7**	**5 → 1**	**2 → 1**
17 eliminadas	Antes vs Después	Variables en memoria	Complejidad: 2 → 1

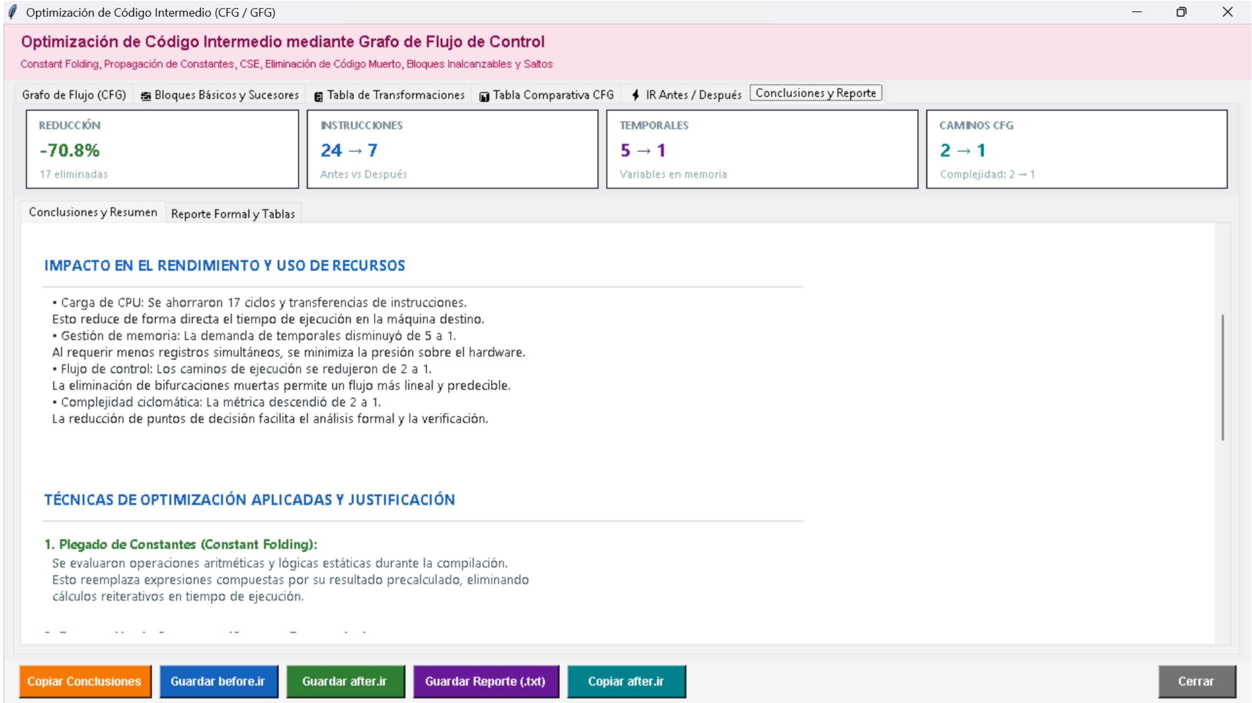
PRESENTACIÓN DE RESULTADOS: OPTIMIZACIÓN DE CÓDIGO INTERMEDIO (IR)

RESUMEN DE LA OPTIMIZACIÓN

- Eficiencia de reducción: Se obtuvo una reducción neta del 70.8% en el código intermedio.
- Instrucciones procesadas: El programa pasó de 24 a 7 instrucciones (17 eliminadas).
- Preservación semántica: El programa resultante mantiene estricta equivalencia funcional con el código original, alcanzando el mismo resultado con menor costo computacional.

IMPACTO EN EL RENDIMIENTO Y USO DE RECURSOS

- Carga de CPU: Se ahorraron 17 ciclos y transferencias de instrucciones. Esto reduce de forma directa el tiempo de ejecución en la máquina destino.
- Gestión de memoria: La demanda de temporales disminuyó de 5 a 1. Al requerir menos registros simultáneos, se minimiza la presión sobre el hardware.


Optimización de Código Intermedio (CFG / GFG)
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Conclusiones y Resumen
Reporte Formal y Tablas
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

Optimización de Código Intermedio mediante Grafo de Flujo de Control

Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos

REDUCCIÓN	INSTRUCCIONES	TEMPORALES	CAMINOS CFG
-70.8%	**24 → 7**	**5 → 1**	**2 → 1**
17 eliminadas	Antes vs Después	Variables en memoria	Complejidad: 2 → 1

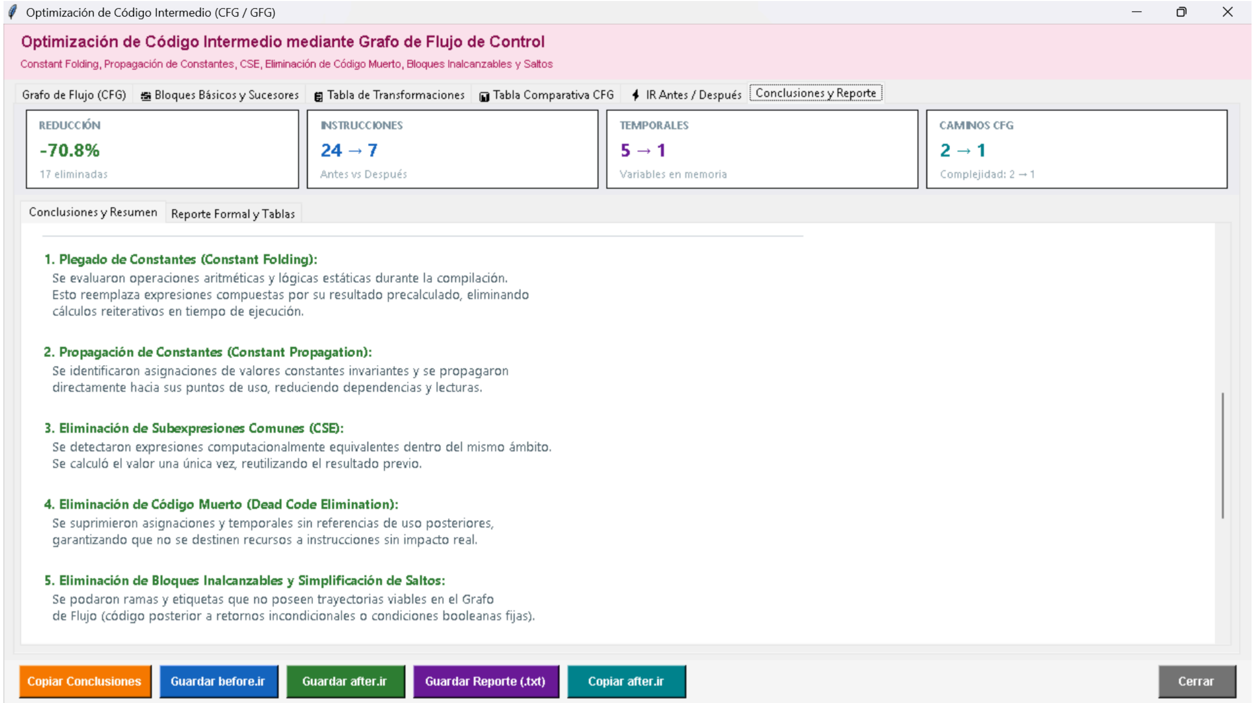
IMPACTO EN EL RENDIMIENTO Y USO DE RECURSOS

- Carga de CPU: Se ahorraron 17 ciclos y transferencias de instrucciones.
Esto reduce de forma directa el tiempo de ejecución en la máquina destino.
- Gestión de memoria: La demanda de temporales disminuyó de 5 a 1.
Al requerir menos registros simultáneos, se minimiza la presión sobre el hardware.
- Flujo de control: Los caminos de ejecución se redujeron de 2 a 1.
La eliminación de bifurcaciones muertas permite un flujo más lineal y predecible.
- Complejidad ciclomática: La métrica descendió de 2 a 1.
La reducción de puntos de decisión facilita el análisis formal y la verificación.

TÉCNICAS DE OPTIMIZACIÓN APLICADAS Y JUSTIFICACIÓN

1. Plegado de Constantes (Constant Folding):

Se evaluaron operaciones aritméticas y lógicas estáticas durante la compilación.
Esto reemplaza expresiones compuestas por su resultado precalculado, eliminando
cálculos reiterativos en tiempo de ejecución.


Optimización de Código Intermedio (CFG / GFG)
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Conclusiones y Resumen
Reporte Formal y Tablas
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

Optimización de Código Intermedio mediante Grafo de Flujo de Control

Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos

REDUCCIÓN	INSTRUCCIONES	TEMPORALES	CAMINOS CFG
-70.8%	24 → 7	5 → 1	2 → 1
17 eliminadas	Antes vs Después	Variables en memoria	Complejidad: 2 → 1

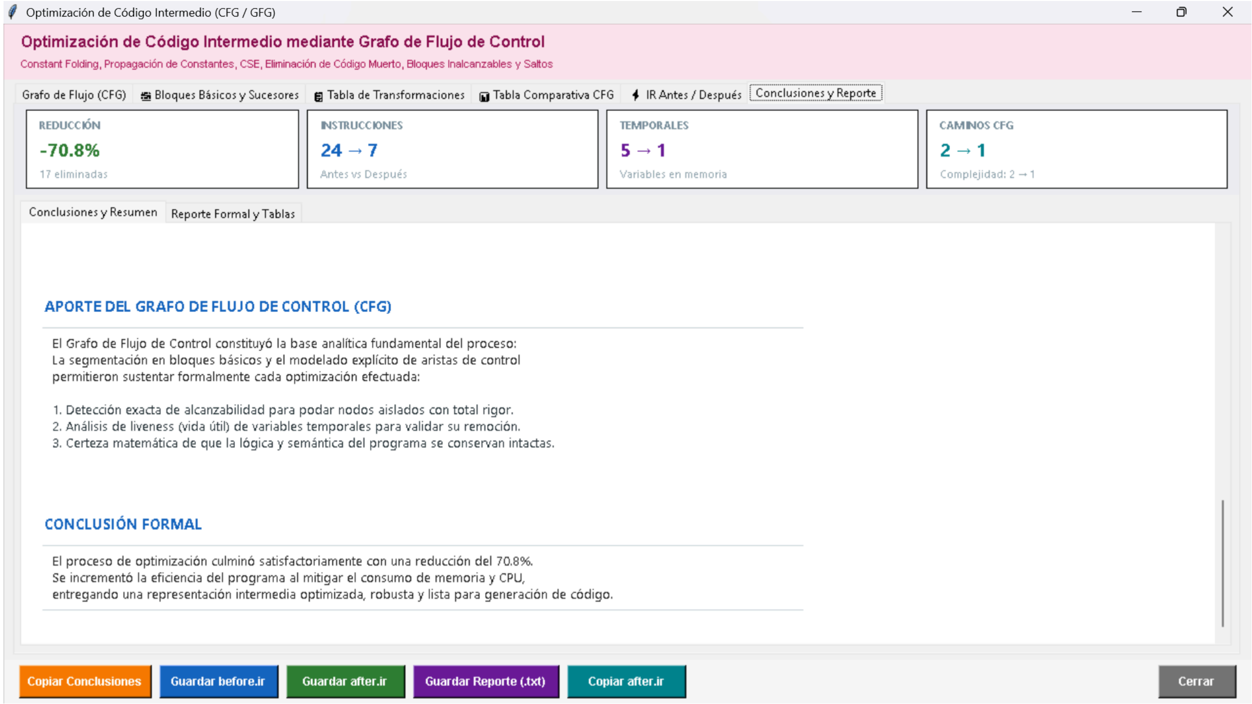
1. Plegado de Constantes (Constant Folding):
Se evaluaron operaciones aritméticas y lógicas estáticas durante la compilación.
Esto reemplaza expresiones compuestas por su resultado precalculado, eliminando
cálculos reiterativos en tiempo de ejecución.

2. Propagación de Constantes (Constant Propagation):
Se identificaron asignaciones de valores constantes invariantes y se propagaron
directamente hacia sus puntos de uso, reduciendo dependencias y lecturas.

3. Eliminación de Subexpresiones Comunes (CSE):
Se detectaron expresiones computacionalmente equivalentes dentro del mismo ámbito.
Se calculó el valor una única vez, reutilizando el resultado previo.

4. Eliminación de Código Muerto (Dead Code Elimination):
Se suprimieron asignaciones y temporales sin referencias de uso posteriores,
garantizando que no se destinen recursos a instrucciones sin impacto real.

5. Eliminación de Bloques Inalcanzables y Simplificación de Saltos:
Se podaron ramas y etiquetas que no poseen trayectorias viables en el Grafo
de Flujo (código posterior a retornos incondicionales o condiciones booleanas fijas).


Optimización de Código Intermedio (CFG / GFG)
Grafo de Flujo (CFG)
Bloques Básicos y Sucesores
Tabla de Transformaciones
Tabla Comparativa CFG
IR Antes / Después
Conclusiones y Reporte
Conclusiones y Resumen
Reporte Formal y Tablas
Copiar Conclusiones
Guardar before.ir
Guardar after.ir
Guardar Reporte (.txt)
Copiar after.ir
Cerrar

Optimización de Código Intermedio mediante Grafo de Flujo de Control

Constant Folding, Propagación de Constantes, CSE, Eliminación de Código Muerto, Bloques Inalcanzables y Saltos

REDUCCIÓN	INSTRUCCIONES	TEMPORALES	CAMINOS CFG
-70.8%	24 → 7	5 → 1	2 → 1
17 eliminadas	Antes vs Después	Variables en memoria	Complejidad: 2 → 1

APORTE DEL GRAFO DE FLUJO DE CONTROL (CFG)

El Grafo de Flujo de Control constituyó la base analítica fundamental del proceso:
La segmentación en bloques básicos y el modelado explícito de aristas de control
permitieron sustentar formalmente cada optimización efectuada:

1. Detección exacta de alcanzabilidad para podar nodos aislados con total rigor.
2. Análisis de liveness (vida útil) de variables temporales para validar su remoción.
3. Certeza matemática de que la lógica y semántica del programa se conservan intactas.

CONCLUSIÓN FORMAL

El proceso de optimización culminó satisfactoriamente con una reducción del 70.8%.
Se incrementó la eficiencia del programa al mitigar el consumo de memoria y CPU,
entregando una representación intermedia optimizada, robusta y lista para generación de código.